Listing 7.6 Register `SeekingAdAdmin` in the Django Admin

```
# RiffMates/content/admin.py
from django.contrib import admin
from django.utils.text import Truncator

from content.models import SeekingAd


@admin.register(SeekingAd)
class SeekingAdAdmin(admin.ModelAdmin):
    list_display = ("id", "date", "owner", "seeking",
        "show_ad", )

    def show_ad(self, obj):
        return Truncator(obj.content).words(5,
            truncate=' ...')
    show_ad.short_description = "Ad"
```

The hidden gem Truncator makes readable shortened text.

Register SeekingAd for use in the Django Admin.

A list of the fields to display, including one calling the custom .show_ad() method

Pass Truncator the ad's content field, restricting the result to 5 words in length, appending " ..." if content got truncated.

The short_description attribute for the method changes the title in the Django Admin listing page.

You've got a new model class, so you need to run the `makemigrations` and `migrate` management commands to perform the necessary database actions. Once you've done that, you can load your development server and add a new classified ad. Play with the various combinations of values for the `seeking`, `musician`, and `band` fields to verify your validation code works.

7.2.3 Writing a ModelForm

It was a bit of a winding road, but you're at your destination. It is time to write the self-serve code allowing users to create a classified ad. `SeekingAd` stores the classified ad, and any web form you write would map almost directly to the storage class. The web form needs an ad type selection widget, musician and band selection widgets, and a place to write the text of the ad. You could write a `Form` class that does all this, but Django's `ModelForm` means you don't have to. A `ModelForm` inherits from the `Form` class and automatically creates the form's fields, based on an associated model.

The `ModelForm` works a bit like a `ModelAdmin`: you create a class and associate it with a model. Unlike `ModelAdmin`, the `ModelForm` uses an inner class called `Meta` to specify its behavior. Inside the inner class, the `model` attribute links the `ModelForm` to an ORM model. The `fields` attribute indicates which fields in the ORM model participate in the form.

If you don't require further customization, then that's all you need. Simply inherit from `ModelForm`, point the inner `Meta` class's `model` attribute at a model, and specify which fields participate. You've got a form. Figure 7.4 shows the equivalent `Form` object that a `ModelForm` represents.